Լաբորատար աշխատանք 11

Աշխատանքի նպատակը. Տարբերակել օբյեկտի փոխանցումը արժեքով, հղումով և հաստատուն հղումով:
Խնդրի պահանջը. Ստեղծել DepositPortfolio դաս՝ պահելով ավանդների գումարների զանգվածը և տարեկան p տոկոսադրույքը: Ժամկետը մեկ տարի է, տոկոսները պարզ են: Հաշվարկներ: Հաշվել ընդհանուր տարեկան եկամուտը՝ Σ(amount·p/100), գտնել ամենամեծ վերջնական գումարով ավանդի ինդեքսը և պորտֆելի միջին գումարից մեծ ավանդների քանակը: Թույլատրելի արժեքներ: բոլոր գումարները >0, 0≤p≤100: Ֆունկցիաներ: analyze(const DepositPortfolio&) արտածում է երեք հաշվարկները: changeCopy(DepositPortfolio object, int index, double value) փոփոխում և արտածում է միայն տեղային պատճենը: reversed(const DepositPortfolio&) վերադարձնում է նոր օբյեկտ՝ շրջված զանգվածով և նույն p-ով: Արտածել բնօրինակը յուրաքանչյուր կանչից հետո: Փորձնական տվյալներ: amount={100000,250000,150000,300000}, p=8: Ինդեքսները սկսվում են 0-ից:
Տեսական մաս. Արժեքով փոխանցումը ստեղծում է պարամետրի պատճեն: Հղումով փոխանցումը թույլ է տալիս փոխել սկզբնական օբյեկտը: const հղումը կիրառվում է առանց պատճենման ընթերցելու համար: Արտաքին սովորական ֆունկցիան private տվյալները ստանում է public մեթոդների միջոցով:

```
#include <iostream>
#include <vector>
#include <cmath>
#include <algorithm>
#include <stdexcept>
using namespace std;
const double EPSILON = 1e-9;
bool isGreater(double a, double b) {
   return (a - b) > EPSILON;
}
class DepositPortfolio {
private:
   vector<double> amounts;
   double p;
public:
   DepositPortfolio(const vector<double>& am, double rate) {
      int n = am.size();
      if (n < 3 || n > 30) {
         throw invalid_argument("Սխալ չափ:");
      }
      if (rate < 0.0 || rate > 100.0) {
         throw invalid_argument("Սխալ տոկոսադրույք:");
      }
      for (double val : am) {
         if (val <= 0.0) {
            throw invalid_argument("Սխալ գումար:");
         }
      }
      amounts = am;
      p = rate;
```

```
    }
    int getSize() const {
        return amounts.size();
    }
    double getP() const {
        return p;
    }
    const vector<double>& getAmounts() const {
        return amounts;
    }
    bool set(int index, double value) {
        if (amounts.size() > 30) {
            return false;
        }
        if (index < 0 || index >= static_cast<int>(amounts.size()) || value <= 0.0) {
            return false;
        }
        amounts[index] = value;
        return true;
    }
    void print() const {
        cout << "Տոկոսադրույք (p): " << p << "%" << endl;
        cout << "Գումարներ: [ ";
        for (size_t i = 0; i < amounts.size(); ++i) {
            cout << amounts[i] << (i + 1 < amounts.size() ? ", " : " ");
        }
        cout << "]" << endl;
    }
};
void analyze(const DepositPortfolio& portfolio) {
    int n = portfolio.getSize();
    double p = portfolio.getP();
    const vector<double>& amounts = portfolio.getAmounts();
    double totalIncome = 0.0;
    double sumAmounts = 0.0;
    for (double amt : amounts) {
        totalIncome += (amt * p / 100.0);
        sumAmounts += amt;
    }
    int maxIndex = 0;
    double maxFinalAmount = amounts[0] + (amounts[0] * p / 100.0);
    for (int i = 1; i < n; ++i) {
        double currentFinalAmount = amounts[i] + (amounts[i] * p / 100.0);
        if (isGreater(currentFinalAmount, maxFinalAmount)) {
            maxFinalAmount = currentFinalAmount;
            maxIndex = i;
        }
    }
    double avgAmount = sumAmounts / n;
    int countAboveAvg = 0;
    for (double amt : amounts) {
        if (isGreater(amt, avgAmount)) {
            countAboveAvg++;
        }
    }
    cout << "Ընդհանուր տարեկան եկամուտ: " << totalIncome << endl;
```

```
    cout << "Ամենամեծ վերջնական գումարով ավանդի ինդեքս: " << maxIndex << " (արժեքը՝ " <<
maxFinalAmount << ")" << endl;
    cout << "Միջին գումարից մեծ ավանդների քանակը: " << countAboveAvg << endl;
}
DepositPortfolio changeCopy(DepositPortfolio object, int index, double value) {
    if (object.set(index, value)) {
        cout << "Տեղային պատճենը փոփոխվեց:" << endl;
        object.print();
        analyze(object);
    } else {
        cout << "Սխալ փոփոխման տվյալներ:" << endl;
    }
    return object;
}
DepositPortfolio reversed(const DepositPortfolio& portfolio) {
    vector<double> revAmounts = portfolio.getAmounts();
    reverse(revAmounts.begin(), revAmounts.end());
    return DepositPortfolio(revAmounts, portfolio.getP());
}
int main() {
    int n;
    cout << "Մուտքագրեք ավանդների քանակը (3-ից 30): ";
    cin >> n;
    vector<double> userAmounts(n);
    cout << "Մուտքագրեք " << n << " ավանդների գումարները (> 0):" << endl;
    for (int i = 0; i < n; ++i) {
        cin >> userAmounts[i];
    }
    double userP;
    cout << "Մուտքագրեք տարեկան տոկոսադրույքը (0-100): ";
    cin >> userP;
    try {
        DepositPortfolio portfolio(userAmounts, userP);

        cout << "\nՍկզբնական պորտֆոլիո:" << endl;
        portfolio.print();
        analyze(portfolio);
        int idx;
        double val;
        cout << "\n Մուտքագրեք ինդեքս և նոր արժեք changeCopy-ի համար: ";
        cin >> idx >> val;
        changeCopy(portfolio, idx, val);
        cout << "\n ChangeCopy-ից հետո:" << endl;
        portfolio.print();
        DepositPortfolio revPortfolio = reversed(portfolio);
        cout << "\n Շրջված նոր օբյեկտ:" << endl;
        revPortfolio.print();
        cout << "\n Reversed-ից հետո:" << endl;
        portfolio.print();
    } catch (const exception& e) {
        cout << e.what() << endl;
    }
    return 0;
}
```